PostgreSQL One-to-One Relationships — Interview Practice

Practice setup

Run the following SQL first. It creates two one-to-one relationships: employees ↔ employee_details and users ↔ user_profiles. The UNIQUE foreign keys enforce that one parent row can have at most one child row.

SQL setup

```
DROP TABLE IF EXISTS employee_details;
DROP TABLE IF EXISTS employees;
DROP TABLE IF EXISTS user_profiles;
DROP TABLE IF EXISTS users;

CREATE TABLE employees (
employee_id SERIAL PRIMARY KEY,
employee_name VARCHAR(100) NOT NULL,
department VARCHAR(50),
salary NUMERIC(10,2) CHECK (salary >= 0)
);

CREATE TABLE employee_details (
employee_id INT PRIMARY KEY,
phone VARCHAR(15) UNIQUE,
address TEXT,
emergency_contact VARCHAR(100),
FOREIGN KEY (employee_id)
REFERENCES employees(employee_id)
ON DELETE CASCADE
);

INSERT INTO employees (employee_name, department, salary) VALUES
('Aarav Sharma', 'AI', 85000),
('Priya Patil', 'Data', 92000),
('Rahul Mehta', 'Backend', 78000),
('Sneha Kulkarni', 'AI', 105000),
('Vikram Joshi', 'Frontend', 68000),
('Neha Shah', 'Data', 99000);

INSERT INTO employee_details (employee_id, phone, address, emergency_contact) VALUES
(1, '9876543210', 'Mumbai', 'Raj Sharma'),
(2, '9876543211', 'Pune', 'Anita Patil'),
(4, '9876543213', 'Nashik', 'Kiran Kulkarni'),
(6, '9876543215', 'Thane', 'Amit Shah');

CREATE TABLE users (
user_id SERIAL PRIMARY KEY,
username VARCHAR(50) UNIQUE NOT NULL,
email VARCHAR(120) UNIQUE NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE user_profiles (
user_id INT PRIMARY KEY,
full_name VARCHAR(100) NOT NULL,
city VARCHAR(50),
experience_years INT CHECK (experience_years >= 0),
FOREIGN KEY (user_id)
REFERENCES users(user_id)
ON DELETE CASCADE
);

INSERT INTO users (username, email) VALUES
('bunny_ai', 'bunny@example.com'),
('priya_data', 'priya@example.com'),
('rahul_dev', 'rahul@example.com'),
```

```
('sneha_ml', 'sneha@example.com');

INSERT INTO user_profiles (user_id, full_name, city, experience_years) VALUES
(1, 'Bunny Sharma', 'Mumbai', 2),
(2, 'Priya Patil', 'Pune', 4),
(4, 'Sneha Kulkarni', 'Nashik', 5);
```

Interview Questions — Do not look for solutions immediately

Q1 — Medium: Complete one-to-one employee information

Write a query that displays every employee with their phone, address, and emergency contact. Employees who do not yet have a row in employee_details must still appear in the result. Return: employee_id, employee_name, department, phone, address, emergency_contact. Order by employee_id.

Q2 — Medium/Hard: Find incomplete one-to-one relationships

Find all employees who do NOT have corresponding records in employee_details. Return employee_id, employee_name, department, and salary. Solve it using an anti-join technique (LEFT JOIN + NULL check).

Q3 — Hard: One-to-one relationship + filtering + aggregation

Find employees whose salary is greater than the average salary of employees who have complete details (phone AND address are not NULL). Return employee_id, employee_name, salary, phone, and address. Sort by salary descending. Constraint: use the one-to-one relationship correctly and avoid accidentally counting duplicate detail rows.

Q4 — Hard: Detect relationship integrity problems

Write a query that identifies whether any employee_details rows have no matching employee. Then write a second query that checks whether the one-to-one rule is violated by duplicate employee_id values. Explain why employee_id being both PRIMARY KEY and FOREIGN KEY prevents duplicates in the current design.

Q5 — Hard: Real interview-style reporting

For each department, return: department, total_employees, employees_with_details, employees_without_details, average_salary_of_employees_with_details. Only include departments having at least 2 employees. Sort by average_salary_of_employees_with_details descending. Important: because this is a one-to-one relationship, your aggregation must not inflate employee counts or salary values.

Interview concepts to master

Concept	What you should know
1:1 relationship	Each parent row maps to at most one child row, and each child row maps to exactly one parent row.
PRIMARY KEY + FOREIGN KEY	Using the child table's foreign key as its PRIMARY KEY enforces at most one child row per parent.
UNIQUE foreign key	A UNIQUE FK is another common way to enforce one-to-one cardinality.
LEFT JOIN	Use it when you must keep all parent rows, including parents without child records.
INNER JOIN	Use it when only records having both parent and child rows should appear.
Anti-join	LEFT JOIN ... WHERE child.key IS NULL is a common interview pattern for missing relationships.
Aggregation risk	Always understand join cardinality before COUNT/AVG/SUM; a one-to-many join can multiply rows, while a true 1:1 join does not.

Interview challenge: Try solving Q1–Q5 without subqueries first where possible. Then see whether you can produce an alternative solution using EXISTS, CTEs, or window functions.